VETRA
PUBLICATION SERIES • DOCUMENT 03

DOC ID: PUB-VETRA-DOC-03
MODULE: AI INTELLIGENCE
DATE: AUGUST 2026

AI SYSTEMS & INTELLIGENCE ARCHITECTURE MONOGRAPH

VETRA

AI Intelligence & Architecture

AI Systems, Intelligence Pipelines & Responsible Automation: Gateway Decoupling, Multi-Agent Orchestration, Governance Safety & Human-in-the-Loop Controls

ARCHITECTED & ENGINEERED BY

Om Rajput

B.Tech in Computer Science & Engineering (Data Science)
SVKM's Institute of Technology, Dhule

AI Architecture Framework

Gateway Pattern • Governance Pipeline • Multi-Agent System • Resilience4j Failover • Redis Caching

Clinical Safety Guarantee

Human-in-the-Loop Validation Required • veterinarian_verified = true Database Invariant

01 — INTELLIGENCE BOUNDARY

AI in Vetra: Philosophy & Operational Role

A strictly auxiliary clinical decision-support subsystem operating under licensed veterinary oversight.

> *"In mission-critical agricultural medicine, AI must never be treated as an autonomous authority. Livestock health decisions carry legal, economic, and biosecurity consequences. In Vetra, AI is architected strictly as an auxiliary diagnostic and triage assistant. The core operating platform remains fully deterministic, while AI-generated suggestions must pass through explicit human verification before mutating official medical records."*
>
> **— Om Rajput, System Architect**

1. Deterministic Core
PostgreSQL 15 and Spring Boot manage authoritative state: animal identities, immutable EVMR records, and spatial outbreak boundaries.

2. AI Auxiliary Intelligence
Multi-agent subsystem parses clinical imagery, estimates triage urgency, suggests differential diagnoses, and drafts structured treatment plans.

3. Human-in-the-Loop
Licensed veterinarians review, modify, or reject AI outputs. Only verified proposals (`veterinarian_verified = true`) enter medical records.

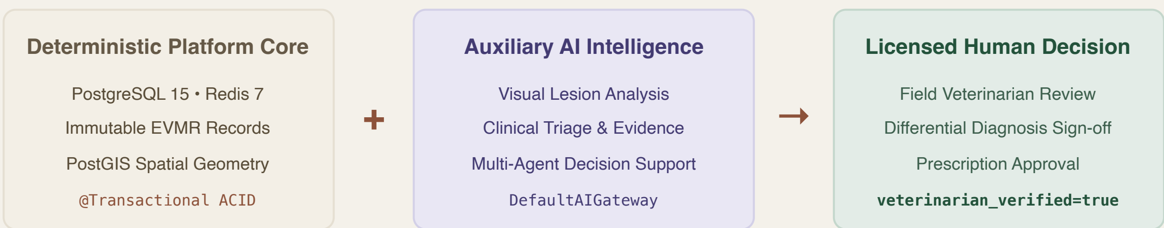

FIGURE 1.1: TRIPARTITE OPERATIONAL RESPONSIBILITY MATRIX IN VETRA PLATFORM

02 — SUBSYSTEM TOPOLOGY

Vetra AI Subsystem Architecture

Decoupled enterprise pipeline bridging mobile clients, governance filters, resilient failover, and clinical workflows.

TIER 1: CLIENT & INGRESS • Flutter Mobile App & Spring Boot REST In
`POST /api/v1/ai/scans • S3 Upload`

TIER 2: AI GATEWAY FACADE • DefaultAIGateway & Prompt Resolution
PromptRegistry • PromptRenderer (Variable Injection) • Capability Matching • SHA-256 Cache Key

TIER 3: ENTERPRISE GOVERNANCE PIPELINE • Pre/Post Execution Controls
AISafetyFilter (Injection/Toxicity) • AIPolicyEngine (Model Allowlist) • AIBudgetManager • AIAuditService

TIER 4: CACHING & RESILIENCE • Redis Cache & FailoverManager
Redis AICacheManager (24h TTL) • Resilience4j Retry & CircuitBreaker • ProviderRouter Dynamic Selection

TIER 5: PROVIDER EXECUTION • GeminiAIProvider & Fallback Providers
`Gemini 1.5 Pro / Flash / NoOp`

FIGURE 2.1: THE PRIMARY VETRA AI SUBSYSTEM & GATEWAY ARCHITECTURE

Pure Orchestration Isolation

Domain services never interact with raw LLM APIs directly. All AI requests pass through `DefaultAIGateway`, ensuring centralized policy enforcement, prompt templating, and audit logging.

Resilient Provider Fallback

If an upstream model endpoint experiences rate limits or timeouts, `FailoverManager` transparently retries or fails over to alternative registered providers without application failure.

03 — GATEWAY DESIGN

The AI Gateway Facade (`DefaultAIGateway`)

Decoupling clinical business logic from external AI SDKs and orchestrating multi-layer execution.

Why the Gateway Exists

- **Zero Vendor Lock-in:** Abstract interface (`AIGateway`) prevents direct dependencies on Google GenAI or third-party SDKs.
- **Centralized Governance:** Evaluates safety, policy, and budget constraints uniformly across all AI features.
- **Performance & Cost Control:** Checks Redis cache with single-flight stampede protection before invoking remote models.

Gateway Responsibilities

1. Resolves `PromptDescriptor` from `PromptRegistry`.
2. Renders variables into templates via `PromptRenderer`.
3. Evaluates enterprise governance in `AIGovernancePipeline`.
4. Computes deterministic SHA-256 cache keys.
5. Delegates provider execution to `FailoverManager`.

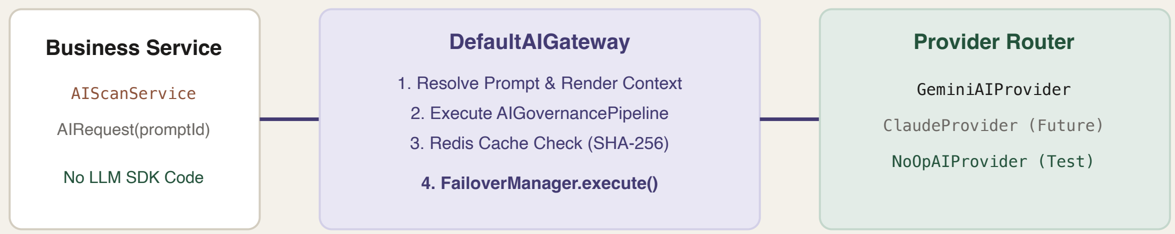

FIGURE 3.1: AI GATEWAY REQUEST DECOUPLING & ORCHESTRATION FLOW

Clean Exception Abstraction

All downstream HTTP or provider exceptions are caught and wrapped into typed domain exceptions (`AITimeoutException`, `AIRateLimitException`, `AIBudgetExceededException`, `AISafetyViolationException`) extending `AIException`.

04 — ENTERPRISE CONTROLS

Enterprise Governance Pipeline (`AIGovernancePipeline`)

Multi-stage pre-flight validation, safety screening, budget quotas, and post-execution accounting.

1. AISafetyFilter
Inspects rendered prompts and inputs for prompt injection, toxic content, and sensitive PII before model dispatch.

2. AIPolicyEngine
Validates tenant authorization for requested capabilities and model tiers against enterprise policy rules.

3. AIBudgetManager
Pre-flight checks on monthly token and dollar budget limits; records actual token usage and cost post-execution.

4. AIAuditService
Emits immutable audit telemetry records capturing prompt ID, tenant, token count, cost, and latency.

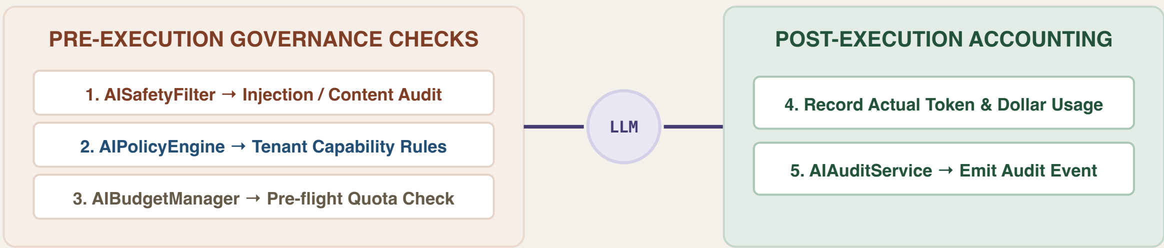

FIGURE 4.1: THE 5-STEP ENTERPRISE AI GOVERNANCE AND ACCOUNTING PIPELINE

Synchronous Pre-Flight Rejection
If a safety policy or token budget is breached, the pipeline immediately aborts execution before calling any external model, throwing `AISafetyViolationException` or `AIBudgetExceededException` (HTTP 422 Unprocessable Entity), saving LLM operational expenses.

05 — SEQUENCE TRACE

End-to-End AI Request Lifecycle

The asynchronous sequence from mobile lesion capture to AI gateway processing and database persistence.

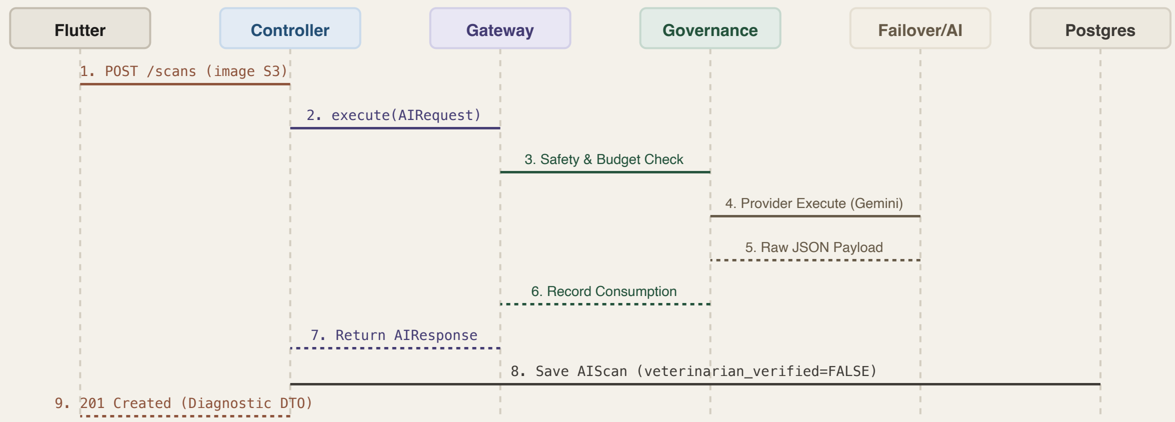

FIGURE 5.1: SYNCHRONOUS EXECUTION SEQUENCE TRACE FOR CLINICAL VISUAL AI SCANS

Immutable S3 Storage Linkage

Photos are uploaded directly to Amazon S3 via presigned URLs before AI scan invocation. The database record references the permanent S3 URI, ensuring an immutable clinical image audit trail.

Default Unverified Status

Every new AI scan record is persisted with `veterinarian_verified = false`. The proposal remains in a pending triage queue until an authenticated veterinarian executes review.

06 — AGENT SPECIALIZATION

Specialized Clinical Multi-Agent Subsystem

Coordinated expert agents dividing complex veterinary tasks into discrete, verifiable reasoning stages.

1. DiagnosisAgent (`DiagnosisAgentImpl`)

- **Input:** Visual lesion images, animal species, age, vital signs, reported clinical symptoms.
- **Task:** Evaluates pathology against veterinary disease database.
- **Output:** Ranked differential diagnoses with confidence scores and clinical rationale.

2. TreatmentAgent (`TreatmentAgentImpl`)

- **Input:** Confirmed/suspected diagnosis, animal weight, lactation state.
- **Task:** Formulates evidence-based therapeutic protocols.
- **Output:** Recommended pharmacological interventions, dosages, and withdrawal periods.

3. KnowledgeAgent (`KnowledgeAgentImpl`)

- **Input:** Clinical query strings and disease keywords.
- **Task:** Vector search against embedded veterinary clinical guidelines.
- **Output:** Verified clinical references and biosecurity protocols.

4. ReportAgent (`ReportAgentImpl`)

- **Input:** Synthesized diagnostic and treatment agent outputs.
- **Task:** Formulates dual role-tailored communication views.
- **Output:** Technical summary for veterinarians; plain-language care guide for farmers.

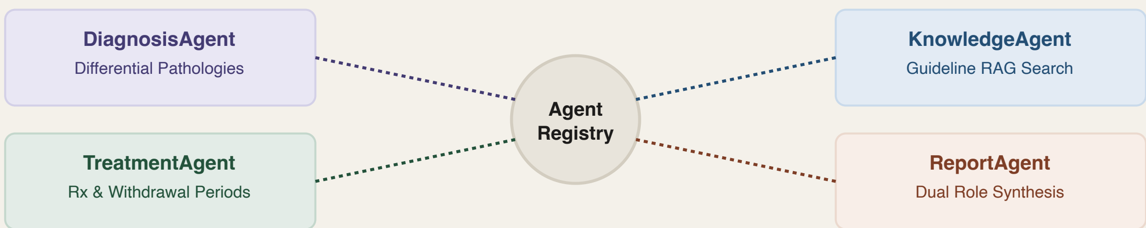

FIGURE 6.1: SPECIALIZED CLINICAL AGENT DIVISION OF LABOR AND REGISTRY COORDINATION

07 — CLINICAL PIPELINE

Clinical Workflow & Triage Engine

Multi-stage pipeline orchestrating evidence aggregation, triage urgency ranking, and care coordination.

1. Evidence Aggregator

Combines farmer observations, vital signs (temp, pulse), laboratory results, and sensor data into an immutable evidence package.

2. Triage Engine

Evaluates clinical acuity into four discrete tiers: `EMERGENT`, `URGENT`, `ROUTINE`, or `ELECTIVE`.

3. Decision Support

Computes explainable diagnostic candidates with uncertainty flags and differential justifications.

4. Action Plan Engine

Generates synchronized care plans: clinical triage for the vet, and quarantine instructions for the farmer.

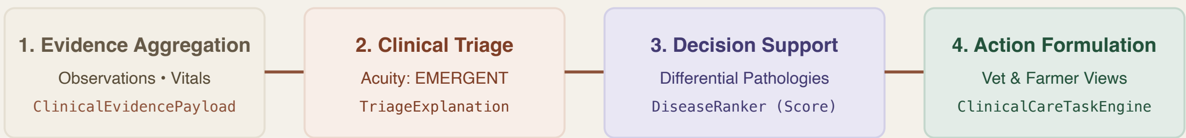

FIGURE 7.1: THE 4-STAGE CLINICAL CASE WORKFLOW & CARE TASK ORCHESTRATION PIPELINE

Care Task Escalation Engine

`CareTaskEscalationEngine` tracks active care assignments. If high-priority triage tasks (e.g. suspected anthrax or hemorrhagic septicemia) remain unacknowledged beyond configured thresholds, automated escalation events alert regional veterinary supervisors.

08 — CONTEXT ASSEMBLY

Prompt Engineering & Context Assembly

Structured prompt templates, variable hydration, and strict context boundary isolation.

Prompt Registry Architecture

- **Template Decoupling:** Prompts are defined as versioned templates in `PromptRegistry` rather than hardcoded strings.
- **Capability Metadata:** Each `PromptDescriptor` declares required capabilities (e.g. `VISION`, `STRUCTURED_OUTPUT`) used by `ProviderRouter`.
- **Hydration:** `PromptRenderer` injects validated animal demographics and clinical variables into template placeholders.

Zero-PII Context Boundary

Prompt templates exclude personal identifiable information (farmer name, phone number, national IDs). The LLM context contains strictly clinical attributes: species, breed, estimated weight, age, observed lesion description, and anonymized vital signs.

Representative Prompt Template: Bovine Lesion Analysis

```
You are an expert board-certified veterinary pathologist assistant. Analyze the provided animal lesion image and clinical context to generate a differential diagnosis. CLINICAL CONTEXT: - Species: {{species}} - Breed: {{breed}} | Age: {{ageMonths}} months | Gender: {{gender}} - Reported Symptoms: {{symptoms}} - Vital Signs: Temp: {{temperatureC}} °C, Pulse: {{pulseBpm}} bpm REQUIREMENTS: 1. Provide top 3 differential diagnoses with estimated confidence [0.0 - 1.0]. 2. Identify primary anatomical lesion characteristics and severity. 3. Highlight mandatory biosecurity warnings if an infectious pathogen is suspected. 4. Output STRICT JSON conforming to the DiagnosticInsightDto schema.
```

Deterministic Template Rendering

`PromptRenderer` validates that all mandatory placeholders are populated before dispatch, throwing `AIConfigurationException` if required variables are missing.

Token Context Budgeting

Prompt lengths are bounded to prevent exceeding model context windows, guaranteeing fast sub-2.5s inference latencies across 4G rural field conditions.

09 — SCHEMA CONTRACTS

Structured Output & Strict Schema Validation

Transforming stochastic model generations into strongly typed, validated Java domain DTOs.

Why Structured Output is Mandatory

Production backend systems cannot process unstructured conversational text. Vetra requires deterministic JSON structures that map directly to Java records and database columns, ensuring validation constraints (`@NotNull`, `@Min`, `@Max`) are verified before domain consumption.

Malformed Response Handling

If an LLM produces truncated JSON or invalid schema types, Jackson parsing fails and throws `AIInvalidResponseException`. `FailoverManager` captures the error and triggers an immediate single-retry or fallbacks to the rule-based clinical engine.

Schema Contract: `DiagnosticInsightDto`

```
public record DiagnosticInsightDto( @NotBlank String primaryDiagnosis, @DecimalMin("0.0") @DecimalMax("1.0") Double
confidenceScore, List<DifferentialCandidate> differentials, String clinicalReasoning, TriageAcuity triageLevel,
List<String> suggestedLabTests, boolean biosecurityAlertRequired ) {}
```

Representative Validated Model Output Payload


```
{ "primaryDiagnosis": "Bovine Papillomatosis", "confidenceScore": 0.88, "differentials": [ { "disease": "Lumpy Skin
Disease", "confidence": 0.65 }, { "disease": "Dermatophilosis", "confidence": 0.32 } ], "clinicalReasoning":
"Multiple proliferative nodular lesions observed along cervical region without systemic ulceration.", "triageLevel":
"ROUTINE", "suggestedLabTests": ["Histopathology biopsy", "PCR viral assay"], "biosecurityAlertRequired": false }
```

10 — CLINICAL SAFETY

AI Safety, Guardrails & Human Verification

Enforcing the `veterinarian_verified` database invariant and content safety boundaries.

The Verification Invariant

In the `ai_scans` relational table, the column `veterinarian_verified` defaults to FALSE. AI suggestions cannot be used to generate official prescriptions, alter animal health status, or initiate quarantine protocols until an authenticated veterinarian reviews and approves the scan.

Prompt Injection Mitigation

`AISafetyFilter` uses strict input sanitation and delimiter boundaries. User-provided symptom strings are wrapped in isolated CDATA-style text blocks, neutralizing adversarial prompt override attempts.

FIGURE 10.1: HUMAN-IN-THE-LOOP VERIFICATION GATE PRIOR TO MEDICAL RECORD INGESTION

Rejection & Overwrite Capabilities

Veterinarians can reject incorrect AI suggestions via `POST /api/v1/ai/scans//reject` or override the diagnosis with their clinical findings. Both the original AI prediction and the veterinarian's final correction are archived for auditability.

11 — AI RESILIENCE

Reliability, Fault Tolerance & Failover

Per-provider Resilience4j circuit breakers, timeout thresholds, and graceful rule-based fallbacks.

1. Timeouts (10s)

Hard time limiter prevents slow external LLM inference from tying up backend Tomcat worker threads.

2. Circuit Breakers

Tracks per-provider failure rates. Opens after 50% failures in a 10-call sliding window to fast-fail traffic.

3. Fallback Providers

On provider exhaustion, routes automatically to `NoOpAIProvider` or deterministic rule engine without throwing 500 errors.

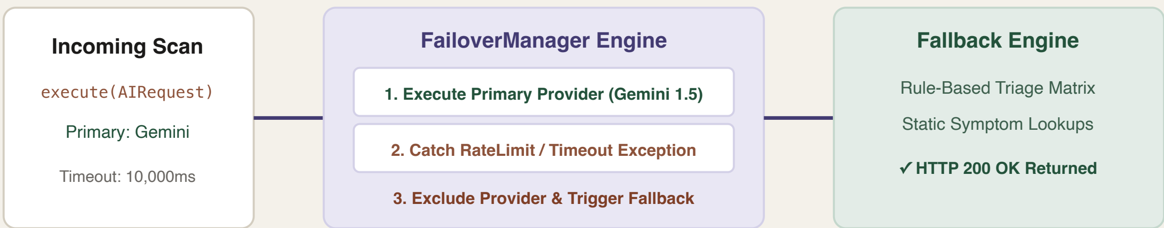

FIGURE 11.1: FAILOVERMANAGER MULTI-TIER RESILIENCE & RULE-BASED FALLBACK FLOW

Graceful Degradation Guarantee

Under total third-party AI provider outage, Vetra degrades gracefully: the platform continues operating all core veterinary functions (passports, appointments, EVMR records) while presenting rule-based diagnostic guidance tagged with `source = FALLBACK_RULES`.

12 — PERFORMANCE & CACHING

Distributed AI Caching & Stampede Protection

Deterministic SHA-256 cache keys, 24-hour TTL policies, and single-flight concurrency locks.

Deterministic Cache Key Generation

`CacheKeyGenerator` computes cryptographic SHA-256 digests over: `promptId`, sorted variable keys/values, image hash (if present), and target model identifier. Identical symptom queries return cached responses in <5ms.

Single-Flight Stampede Protection

When multiple concurrent requests query the same un-cached clinical context simultaneously, `AICacheManager` acquires a Redis mutex lock for that key. Only one request invokes the LLM API; other threads await the single cached result.

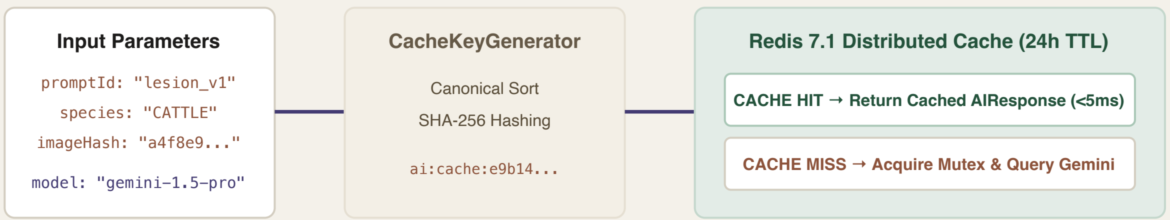

FIGURE 12.1: DETERMINISTIC SHA-256 KEY GENERATION AND SINGLE-FLIGHT CACHING FLOW

13 — DATA GOVERNANCE

Authoritative Domain Data vs. AI Output

Strict architectural isolation between deterministic database truth and probabilistic machine predictions.

Authoritative Relational Truth

PostgreSQL tables (`animals`, `appointments`, `medical_records`, `disease_reports`) hold legal, clinical, and biosecurity truth. These records are created strictly by authenticated users, signed with transactional ACID guarantees, and protected by optimistic locking.

Probabilistic AI Proposals

AI outputs (diagnostic insights, triage scores, care tasks) are stored in dedicated auxiliary tables (`ai_scans`, `clinical_cases`) as **candidate proposals**. They never overwrite or mutate core animal records directly.

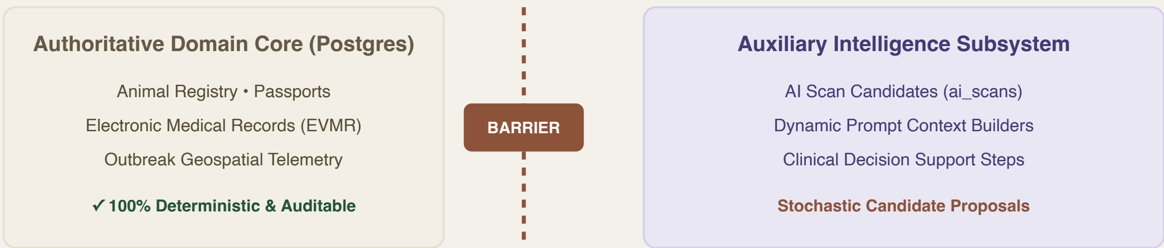

FIGURE 13.1: STRICT BOUNDARY BETWEEN AUTHORITATIVE CORE DATA AND AI INTELLIGENCE

Unidirectional Read Flow

AI prompt builders are permitted to read authoritative animal and demographic data to hydrate context. However, AI responses can **never write directly** to core domain tables—they write exclusively to candidate triage entities until signed by a human.

14 — INTELLIGENCE TELEMETRY

AI Observability, Metrics & Distributed Tracing

Real-time token counting, inference latency tracking, and OpenTelemetry child spans.

Micrometer AI Metrics

`ai_requests_total`, `ai_inference_duration_seconds`, and `ai_token_usage_total` tagged by promptId, tenantId, and modelId.

Distributed Tracing Spans

Child spans created for `AIGateway.execute`, `AISafetyFilter.evaluate`, and `AIProvider.generate` with OpenTelemetry propagation.

Token & Cost Telemetry

`AIAuditService` logs prompt and completion token counts and estimated costs per inference call for financial accountability.

```
Root Span: POST /api/v1/ai/scans (traceId=c5946b...)
  Child Span: DefaultAIGateway.execute (promptId=bovine_lesion_v1)
    Span: AISafetyFilter.evaluate (0.4ms)    Span: GeminiAIProvider.generate (1,842ms)
    Span: AIAuditService.recordAuditEvent (tokens=412, cost=$0.0006)
```

FIGURE 14.1: OPENTELEMETRY TRACE SPAN HIERARCHY ACROSS THE AI EXECUTION GATEWAY

15 — AI QUALITY ENGINEERING

Testing, Mocking & Verification Strategy

70+ automated AI test cases covering prompt rendering, circuit breaker failover, and governance.

TEST SUITE LAYER	TARGET COMPONENTS & SCOPE	MOCKING & TECHNOLOGY	RESULT
Gateway & Governance	`DefaultAIGatewayTest`, `AIGovernanceIntegrationTest`, `AISafetyFilterTest`.	JUnit 5, Mockito, In-Memory Registry	VERIFIED
Failover & Circuit Breakers	`FailoverManagerTest`, rate limit recovery, provider exhaustion simulations.	Resilience4j In-Memory Registry	VERIFIED
Multi-Agent Implementations	`DiagnosisAgentTest`, `TreatmentAgentTest`, `ReportAgentTest`.	Mocked AI Gateway Responses	VERIFIED
Clinical Workflow Engine	`ClinicalWorkflowEngineTest`, `ClinicalTriageEngineTest`.	Rule Matrix & State Validation	VERIFIED
Prompt & Cache Testing	`PromptRendererTest`, `AICacheManagerTest`, SHA-256 key stability.	In-Memory Cache & Template Assertions	VERIFIED

Deterministic Unit Verification

Unit and integration test suites execute against `NoOpAIProvider` and mocked gateway fixtures, verifying that all schema deserialization, circuit breakers, and error mapping execute deterministically with 0 external API costs.

Formal Evaluation Note (Honest Status)

Deterministic schema compliance and resilience failover are 100% verified. A formal statistical benchmark suite evaluating model precision against annotated clinical datasets is identified as a planned future phase.

16 — ARCHITECTURAL RIGOR

Key Engineering Decisions & Real Limitations

Technical rationales, architectural trade-offs, and an honest assessment of current subsystem boundaries.

1. Gateway Pattern over Direct SDKs

RATIONALE: Direct SDK calls cause vendor lock-in and chaotic governance. The Gateway pattern centralizes prompt templates, caching, and failover.
TRADE-OFF: Introduces internal gateway interface abstractions and extra layers.

2. Human Verification Invariant

RATIONALE: Prevents AI hallucinations from contaminating legal medical histories or triggering incorrect antibiotics.
TRADE-OFF: Requires licensed veterinarian time for approval; not fully autonomous.

Explicit Subsystem Limitations & Boundary Analysis

1. Network Latency
Remote multimodal LLM inference takes 1.5–3.0s over cloud endpoints, requiring asynchronous loading indicators in mobile UI.

2. Novel Disease Hallucination
Rare or localized tropical livestock infections may produce lower confidence scores, requiring mandatory clinical review.

3. Cloud Dependency
Multimodal vision models currently execute in cloud endpoints; on-device edge model execution is planned for Phase 2.

ENGINEERING PUBLICATION SERIES • DOCUMENT 03

VETRA: AI Intelligence & Architecture

AI Systems, Intelligence Pipelines & Responsible Automation

Built & Engineered by Om Rajput
B.Tech in Computer Science & Engineering (Data Science)
SVKM's Institute of Technology, Dhule

"Engineering responsible auxiliary intelligence with strict human clinical oversight."